

GALGOTIAS UNIVERSITY

# Soft Computing Capstone Beginner Project Start Workbook

A step-by-step guide for students who are new to GitHub, datasets, algorithms, and software projects.

Use this workbook before writing the main algorithm.

Team size: 3-4 students

M1: finish one week before the mid-semester examination

M2: finish one week before the end-semester examination

# How to use this workbook

**You are not expected to know everything on day one.** Follow one small action at a time. Save evidence after every action. Ask for faculty review after Step 1, before building the main algorithm.

## For every project

|   |                                                                          |
|---|--------------------------------------------------------------------------|
| 1 | Read the complete project specification once. Do not code while reading. |
| 2 | Read the two-page section for your project in this workbook.             |
| 3 | Create the repository and Step 1 files.                                  |
| 4 | Prepare only 20 clean sample rows first.                                 |
| 5 | Write the simple baseline and five test cases.                           |
| 6 | Run the validation program and correct every error.                      |
| 7 | Show the Step 1 checklist to faculty.                                    |
| 8 | After approval, continue with Steps 2-5 for M1 and Steps 6-8 for M2.     |

## Important words

| Word           | Simple meaning                                                                       |
|----------------|--------------------------------------------------------------------------------------|
| Repository     | The main project folder stored on GitHub.                                            |
| Commit         | A saved checkpoint with a short message explaining what changed.                     |
| Dataset        | A table of examples used to build or test the project.                               |
| Scenario       | One possible situation given to the program, such as a hot occupied room.            |
| Baseline       | A simple method used for comparison. Your advanced method should improve on it.      |
| Validation     | Checks that data is present, correctly named, and inside allowed limits.             |
| Input / output | Inputs go into the program. Outputs are the decisions or predictions returned by it. |

## Eight steps from start to final product

| Step | Stage | Work                            | Meaning                                                                                                                              |
|------|-------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| 1    | M1    | Freeze the project contract     | Choose the exact user, problem, inputs, outputs, units, baseline, data source, five test cases, repository, and responsibilities.    |
| 2    | M1    | Build the data pipeline         | Collect or generate 10,000+ cited records/scenarios. Clean, validate, prepare, and split them so another person can repeat the work. |
| 3    | M1    | Implement the baseline          | Build the simple comparison method and save its results.                                                                             |
| 4    | M1    | Implement the main M1 technique | Make the required fuzzy, ANN, or GA method work and show its important internal steps.                                               |
| 5    | M1    | Assemble Product V1             | Connect input, validation, baseline, main method, output, and at least two visuals in one runnable screen.                           |
| 6    | M2    | Add the advanced technique      | Add optimization, a hybrid model, more algorithms, priorities, or multiple objectives.                                               |
| 7    | M2    | Experiment and stress-test      | Compare baseline, M1, and M2. Change parameters and test noise, extremes, unseen cases, and impossible constraints.                  |
| 8    | M2    | Finish Product V2               | Improve and deploy the app. Finish the report, video, presentation, README, and viva preparation.                                    |

**M1 is not a proposal.** Product V1 must accept input, perform processing, and show useful output. **M2 is not only a prettier screen.** Product V2 must contain a real technical improvement, experiments, deployment, and final evidence.

## STEP 1 FOR EVERY TEAM

# Set up the project correctly

## A. Create the repository

Use a clear repository name, such as **SC01-Climate-Controller-Team-A**. Add every team member as a collaborator. Create the structure below.

```
project-repository/  
|-- README.md  
|-- requirements.txt  
|-- docs/  
| `-- STEP-1.md  
|-- data/  
| |-- README.md  
| `-- sample_input.csv  
|-- notebooks/  
|-- src/  
|-- app/  
|-- results/  
`-- report/
```

## B. Give every member a first responsibility

| Responsibility                   | First job                                                                             |
|----------------------------------|---------------------------------------------------------------------------------------|
| Repository and integration       | Create folders, add members, and keep the run instructions correct.                   |
| Data and scenarios               | Define fields and units, create the 20-row sample, and write data-source information. |
| Baseline and algorithm           | Write the simple baseline and draw the future algorithm flow.                         |
| Interface, testing, and evidence | Draw the screen, write five test cases, and save screenshots/results.                 |

For a three-person team, one person may combine repository work with interface/testing. Everyone must still understand the complete project.

## C. Put these headings in docs/STEP-1.md

```
# Step 1 Project Contract  
## Team and responsibilities  
## One-sentence problem  
## User of the product  
## Inputs and units  
## Outputs and units  
## Baseline method  
## Soft Computing method for M1  
## Advanced method for M2  
## Dataset/scenario sources  
## Five mandatory test cases  
## Product V1 screen sketch  
## Risks and assumptions  
## Step 1 completion evidence
```

# Prepare evidence before the main algorithm

## D. Make an input and output dictionary

For every field, write its name, simple meaning, type, unit, allowed range or category, missing-value rule, data source, and whether it is an input, target, or output.

## E. Make a small 20-row sample

|   |                                                         |
|---|---------------------------------------------------------|
| 1 | Create the correct column headings.                     |
| 2 | Add ordinary rows.                                      |
| 3 | Add boundary rows near the minimum and maximum.         |
| 4 | Add difficult or contradictory rows.                    |
| 5 | Add one deliberately wrong row in a separate test file. |
| 6 | Run validation and correct every unexpected error.      |

## F. Write the baseline before the advanced method

The baseline is a simple rule, threshold, greedy method, or average. It gives you something fair to compare with the fuzzy, ANN, or GA solution.

## G. Draw one Product V1 screen

|                          |                                   |
|--------------------------|-----------------------------------|
| <input type="checkbox"/> | Input area                        |
| <input type="checkbox"/> | Run / Recommend / Optimize button |
| <input type="checkbox"/> | Baseline output                   |
| <input type="checkbox"/> | Soft Computing output             |
| <input type="checkbox"/> | One explanation or chart          |
| <input type="checkbox"/> | Validation and error-message area |

## H. Save work with meaningful commits

Good messages: **Define input fields and units; Add 20 validated scenarios; Document baseline rules; Add invalid-input test.**

Weak messages: **update; final; changes; project complete.**

## Common beginner mistakes

|                          |                                                                 |
|--------------------------|-----------------------------------------------------------------|
| <input type="checkbox"/> | Do not copy code that nobody in the team can explain.           |
| <input type="checkbox"/> | Do not download data without saving its URL and licence.        |
| <input type="checkbox"/> | Do not create 10,000 bad rows before 20 rows work correctly.    |
| <input type="checkbox"/> | Do not build only slides or a report for M1.                    |
| <input type="checkbox"/> | Do not wait until the last day to create the GitHub repository. |
| <input type="checkbox"/> | Do not let one member upload all work in one commit.            |

# Intelligent Campus Climate and Energy Controller

**In simple words:** Build a small app that looks at room conditions and suggests how much cooling and fan speed are needed. It should also avoid wasting energy when the room is empty or electricity is expensive.

## What you must finish

|                                |                                                                                                               |
|--------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>M1 - Working Product V1</b> | A fuzzy controller for one room, compared with a simple temperature-rule controller.                          |
| <b>M2 - Better Product V2</b>  | Use a Genetic Algorithm (GA) to improve the fuzzy settings, test difficult weather cases, and deploy the app. |

## Your eight steps

|   |                                                      |
|---|------------------------------------------------------|
| 1 | Choose one room and define its inputs and outputs.   |
| 2 | Create and validate climate and occupancy scenarios. |
| 3 | Build a simple fixed-threshold controller.           |
| 4 | Build the fuzzy controller and explain its rules.    |
| 5 | Join everything in a working Product V1 screen.      |
| 6 | Use GA to tune membership functions or rules.        |
| 7 | Compare fixed, fuzzy, and GA-fuzzy results.          |
| 8 | Deploy Product V2 and finish all evidence.           |

**Do not jump ahead.** First complete Step 1 below. Do not start the main fuzzy, ANN, or GA code until the team can show the Step 1 evidence.

# Do this first

A beginner team can finish this in three to five working days. Divide the work, but discuss every item together.

## 1. Create these files

- ☐ Create `data/sample_input.csv`.
- ☐ Create `data/README.md`.
- ☐ Create `docs/STEP-1.md`.
- ☐ Create `src/validate_data.py`.

## 2. Put these columns in the sample data

**Columns:** `scenario_id`, `temperature_c`, `humidity_pct`, `occupancy_count`, `tariff_level`

**Starter rules:** Temperature 18-45 C; humidity 20-100%; occupancy 0 to room capacity; tariff low, medium, or high.

## 3. Write the expected output

`cooling_pct` (0-100), `fan_level` (off/low/medium/high), `energy_action` (normal/save/avoid\_peak)

## 4. Write the simple baseline

Use simple temperature bands for cooling. Increase fan speed when humidity is high. If occupancy is zero, use minimum operation.

## 5. Create exactly 20 sample rows

Begin with only 20 rows. Include normal, boundary, and difficult rows. Your validation program must print the file shape, column names, missing-value count, and invalid values. After this works, you may plan the required 10,000 or more records/scenarios.

## 6. Include these five test cases

- ☐ Comfortable occupied room
- ☐ Hot and humid occupied room
- ☐ Empty room
- ☐ Heat wave with high tariff
- ☐ Cool room with high humidity

## 7. Make small Git commits

Example commit: **Define climate input schema and five test cases**

Every member must make at least one meaningful commit. Do not upload the complete project in one final commit.

## Stop and show this to faculty

|                                                                                           |                                                                                    |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <input type="checkbox"/> Repository works; all members have access.                       | <input type="checkbox"/> STEP-1 explains the problem, inputs, outputs, M1, and M2. |
| <input type="checkbox"/> Field names, units, ranges, and missing-value rules are written. | <input type="checkbox"/> The 20-row sample loads without errors.                   |
| <input type="checkbox"/> One deliberately wrong row is rejected.                          | <input type="checkbox"/> Five named test cases are present.                        |
| <input type="checkbox"/> Baseline rules and a Product V1 screen sketch are included.      | <input type="checkbox"/> Every member can run and explain the validation program.  |

# Smart Washing Machine Decision and Resource Optimizer

**In simple words:** Build an app that recommends wash time, water, detergent, and spin speed after the user describes the clothes and water availability.

## What you must finish

|                                |                                                                                               |
|--------------------------------|-----------------------------------------------------------------------------------------------|
| <b>M1 - Working Product V1</b> | A fuzzy washing controller, compared with fixed washing presets.                              |
| <b>M2 - Better Product V2</b>  | Use GA to balance washing quality with water, detergent, and energy use, then deploy the app. |

## Your eight steps

|   |                                                        |
|---|--------------------------------------------------------|
| 1 | Choose the washing machine capacity and define inputs. |
| 2 | Create and validate realistic washing scenarios.       |
| 3 | Build fixed wash-cycle presets.                        |
| 4 | Build fuzzy variables, rules, and outputs.             |
| 5 | Join the controller and screen as Product V1.          |
| 6 | Use GA to tune quality and resource trade-offs.        |
| 7 | Test delicate, scarce-water, and heavy-load cases.     |
| 8 | Deploy Product V2 and finish all evidence.             |

**Do not jump ahead.** First complete Step 1 below. Do not start the main fuzzy, ANN, or GA code until the team can show the Step 1 evidence.

# Do this first

A beginner team can finish this in three to five working days. Divide the work, but discuss every item together.

## 1. Create these files

- ☐ Create **data/sample\_input.csv**.
- ☐ Create **data/README.md**.
- ☐ Create **docs/STEP-1.md**.
- ☐ Create **src/validate\_data.py**.

## 2. Put these columns in the sample data

**Columns:** scenario\_id, dirt\_score, load\_kg, fabric\_type, water\_availability\_pct

**Starter rules:** Dirt score 0-10; load 0.5 kg to machine capacity; fabric delicate/cotton/synthetic/heavy; water 0-100%.

## 3. Write the expected output

wash\_time\_min, water\_litre, detergent\_ml, spin\_rpm

## 4. Write the simple baseline

Create one fixed preset for every fabric type. The preset does not change with dirt or water availability.

## 5. Create exactly 20 sample rows

Begin with only 20 rows. Include normal, boundary, and difficult rows. Your validation program must print the file shape, column names, missing-value count, and invalid values. After this works, you may plan the required 10,000 or more records/scenarios.

## 6. Include these five test cases

- ☐ Lightly soiled half-load
- ☐ Heavily soiled full cotton load
- ☐ Delicate fabric
- ☐ Low-water situation
- ☐ Overloaded input that must be rejected

## 7. Make small Git commits

Example commit: **Add validated sample wash scenarios**

Every member must make at least one meaningful commit. Do not upload the complete project in one final commit.

## Stop and show this to faculty

|                                                                                           |                                                                                    |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <input type="checkbox"/> Repository works; all members have access.                       | <input type="checkbox"/> STEP-1 explains the problem, inputs, outputs, M1, and M2. |
| <input type="checkbox"/> Field names, units, ranges, and missing-value rules are written. | <input type="checkbox"/> The 20-row sample loads without errors.                   |
| <input type="checkbox"/> One deliberately wrong row is rejected.                          | <input type="checkbox"/> Five named test cases are present.                        |
| <input type="checkbox"/> Baseline rules and a Product V1 screen sketch are included.      | <input type="checkbox"/> Every member can run and explain the validation program.  |

# Student Performance and Academic Intervention System

**In simple words:** Build a decision-support app that identifies a student's academic risk level and suggests a helpful intervention. It must use anonymous data and must not punish a student automatically.

## What you must finish

|                                |                                                                                                          |
|--------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>M1 - Working Product V1</b> | One transparent neural classifier and a simple score baseline, shown in a risk and intervention screen.  |
| <b>M2 - Better Product V2</b>  | Compare Perceptron, ADALINE, and Backpropagation; study errors, imbalance, fairness, and deploy the app. |

## Your eight steps

|   |                                                          |
|---|----------------------------------------------------------|
| 1 | Define anonymous inputs, risk labels, and interventions. |
| 2 | Prepare and validate a cited student dataset.            |
| 3 | Build a score-threshold or majority baseline.            |
| 4 | Build one neural classifier and show its learning.       |
| 5 | Create the risk and intervention Product V1.             |
| 6 | Add and compare the other neural methods.                |
| 7 | Study errors, imbalance, noise, and fairness.            |
| 8 | Deploy Product V2 and finish all evidence.               |

**Do not jump ahead.** First complete Step 1 below. Do not start the main fuzzy, ANN, or GA code until the team can show the Step 1 evidence.

# Do this first

A beginner team can finish this in three to five working days. Divide the work, but discuss every item together.

## 1. Create these files

☐ Create **data/sample\_input.csv**.

☐ Create **data/README.md**.

☐ Create **docs/STEP-1.md**.

☐ Create **src/validate\_data.py**.

## 2. Put these columns in the sample data

**Columns:** record\_id, attendance\_pct, assessment\_pct, assignment\_pct, engagement\_score, prior\_performance\_pct, risk\_label

**Starter rules:** Percentages 0-100; engagement 0-10; risk label low, medium, or high. Do not store names or phone numbers.

## 3. Write the expected output

risk category and one clear intervention, such as monitor, mentor, or faculty follow-up

## 4. Write the simple baseline

Use a visible weighted score and simple thresholds. Write down the temporary weights and explain that they must be tested.

## 5. Create exactly 20 sample rows

Begin with only 20 rows. Include normal, boundary, and difficult rows. Your validation program must print the file shape, column names, missing-value count, and invalid values. After this works, you may plan the required 10,000 or more records/scenarios.

## 6. Include these five test cases

☐ Consistently strong student

☐ Low attendance only

☐ Weak marks and assignments

☐ High engagement but weak prior performance

☐ Contradictory or boundary record

## 7. Make small Git commits

Example commit: **Document anonymous academic-risk schema**

Every member must make at least one meaningful commit. Do not upload the complete project in one final commit.

## Stop and show this to faculty

|                                                                                           |                                                                                    |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <input type="checkbox"/> Repository works; all members have access.                       | <input type="checkbox"/> STEP-1 explains the problem, inputs, outputs, M1, and M2. |
| <input type="checkbox"/> Field names, units, ranges, and missing-value rules are written. | <input type="checkbox"/> The 20-row sample loads without errors.                   |
| <input type="checkbox"/> One deliberately wrong row is rejected.                          | <input type="checkbox"/> Five named test cases are present.                        |
| <input type="checkbox"/> Baseline rules and a Product V1 screen sketch are included.      | <input type="checkbox"/> Every member can run and explain the validation program.  |

# Adaptive Traffic Signal Optimization System

**In simple words:** Build a small traffic-junction simulator. The app should choose which direction gets green and for how long, using queue and waiting-time information.

## What you must finish

|                                |                                                                                             |
|--------------------------------|---------------------------------------------------------------------------------------------|
| <b>M1 - Working Product V1</b> | A fuzzy controller for one two-phase junction, compared with a fixed-time signal.           |
| <b>M2 - Better Product V2</b>  | Use GA to tune the controller, test emergencies and heavy queues, and deploy the simulator. |

## Your eight steps

|   |                                                 |
|---|-------------------------------------------------|
| 1 | Define one simple two-phase junction.           |
| 2 | Generate and validate traffic states.           |
| 3 | Build the fixed-time signal and queue updates.  |
| 4 | Build fuzzy phase priority and green duration.  |
| 5 | Create the working junction Product V1.         |
| 6 | Use GA to tune controller parameters.           |
| 7 | Compare controllers in difficult traffic cases. |
| 8 | Deploy Product V2 and finish all evidence.      |

**Do not jump ahead.** First complete Step 1 below. Do not start the main fuzzy, ANN, or GA code until the team can show the Step 1 evidence.

# Do this first

A beginner team can finish this in three to five working days. Divide the work, but discuss every item together.

## 1. Create these files

- ☐ Create `data/sample_input.csv`.
- ☐ Create `data/README.md`.
- ☐ Create `docs/STEP-1.md`.
- ☐ Create `src/validate_data.py`.

## 2. Put these columns in the sample data

**Columns:** `state_id`, `queue_ns`, `queue_ew`, `density_ns_pct`, `density_ew_pct`, `max_wait_ns_sec`, `max_wait_ew_sec`, `emergency_direction`

**Starter rules:** Queues cannot be negative; density 0-100%; wait 0-300 seconds; emergency direction none, NS, or EW.

## 3. Write the expected output

`next_phase` (NS/EW) and `green_duration_sec`, with safe minimum and maximum values

## 4. Write the simple baseline

Alternate north-south and east-west using a fixed 30-second green time.

## 5. Create exactly 20 sample rows

Begin with only 20 rows. Include normal, boundary, and difficult rows. Your validation program must print the file shape, column names, missing-value count, and invalid values. After this works, you may plan the required 10,000 or more records/scenarios.

## 6. Include these five test cases

- ☐ Balanced low traffic
- ☐ Heavy north-south queue
- ☐ Heavy east-west queue
- ☐ Long wait but shorter queue
- ☐ Emergency vehicle override

## 7. Make small Git commits

Example commit: **Document fixed-time traffic baseline**

Every member must make at least one meaningful commit. Do not upload the complete project in one final commit.

## Stop and show this to faculty

|                                                                                           |                                                                                    |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <input type="checkbox"/> Repository works; all members have access.                       | <input type="checkbox"/> STEP-1 explains the problem, inputs, outputs, M1, and M2. |
| <input type="checkbox"/> Field names, units, ranges, and missing-value rules are written. | <input type="checkbox"/> The 20-row sample loads without errors.                   |
| <input type="checkbox"/> One deliberately wrong row is rejected.                          | <input type="checkbox"/> Five named test cases are present.                        |
| <input type="checkbox"/> Baseline rules and a Product V1 screen sketch are included.      | <input type="checkbox"/> Every member can run and explain the validation program.  |

# University Timetable Optimization Engine

**In simple words:** Build a program that places courses into rooms and time slots without clashes. It should also try to reduce gaps and respect preferences.

## What you must finish

|                                |                                                                                                |
|--------------------------------|------------------------------------------------------------------------------------------------|
| <b>M1 - Working Product V1</b> | A Genetic Algorithm timetable for a small test case, compared with random or greedy placement. |
| <b>M2 - Better Product V2</b>  | Add more objectives and constraints, test larger cases, improve the interface, and deploy it.  |

## Your eight steps

|   |                                                         |
|---|---------------------------------------------------------|
| 1 | Define courses, rooms, faculty, slots, and constraints. |
| 2 | Create reusable timetable fixtures and variations.      |
| 3 | Build a random or greedy scheduler.                     |
| 4 | Build the GA and a repair or penalty method.            |
| 5 | Create a timetable viewer as Product V1.                |
| 6 | Add multi-objective constraints.                        |
| 7 | Compare GA settings at different problem sizes.         |
| 8 | Deploy Product V2 and finish all evidence.              |

**Do not jump ahead.** First complete Step 1 below. Do not start the main fuzzy, ANN, or GA code until the team can show the Step 1 evidence.

# Do this first

A beginner team can finish this in three to five working days. Divide the work, but discuss every item together.

## 1. Create these files

- ☐ Create **data/courses.csv**.
- ☐ Create **data/rooms.csv**.
- ☐ Create **data/timeslots.csv**.
- ☐ Create **data/constraints.md**.
- ☐ Create **src/validate\_data.py**.

## 2. Put these columns in the sample data

**Columns:** courses: course\_id, cohort\_id, faculty\_id, weekly\_sessions, duration\_slots, required\_room\_type; rooms: room\_id, capacity, room\_type; timeslots: slot\_id, day, start\_time, end\_time

**Starter rules:** Start with 5 courses, 3 rooms, 3 faculty members, 2 cohorts, and 10 time slots.

## 3. Write the expected output

course\_id, room\_id, slot\_id, faculty\_id, cohort\_id, plus hard-violation and soft-penalty counts

## 4. Write the simple baseline

Place each course in the first available valid slot, or use random placement with retry.

## 5. Create exactly 20 sample rows

Begin with only 20 rows. Include normal, boundary, and difficult rows. Your validation program must print the file shape, column names, missing-value count, and invalid values. After this works, you may plan the required 10,000 or more records/scenarios.

## 6. Include these five test cases

- ☐ Feasible timetable
- ☐ Room shortage
- ☐ Faculty clash
- ☐ Cohort clash
- ☐ Preference conflict with no hard violation

## 7. Make small Git commits

Example commit: **Add timetable entities and clash fixtures**

Every member must make at least one meaningful commit. Do not upload the complete project in one final commit.

## Stop and show this to faculty

|                                                                                           |                                                                                    |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <input type="checkbox"/> Repository works; all members have access.                       | <input type="checkbox"/> STEP-1 explains the problem, inputs, outputs, M1, and M2. |
| <input type="checkbox"/> Field names, units, ranges, and missing-value rules are written. | <input type="checkbox"/> The 20-row sample loads without errors.                   |
| <input type="checkbox"/> One deliberately wrong row is rejected.                          | <input type="checkbox"/> Five named test cases are present.                        |
| <input type="checkbox"/> Baseline rules and a Product V1 screen sketch are included.      | <input type="checkbox"/> Every member can run and explain the validation program.  |

# Multi-Stop Campus Shuttle or Delivery Route Optimizer

**In simple words:** Build an app that finds a good order for visiting campus stops. It should reduce distance or time while respecting priorities and later time windows.

## What you must finish

|                                |                                                                                           |
|--------------------------------|-------------------------------------------------------------------------------------------|
| <b>M1 - Working Product V1</b> | A permutation GA route, compared with the nearest-neighbour method, with a route display. |
| <b>M2 - Better Product V2</b>  | Add traffic, priorities, and time windows; compare methods; and deploy the app.           |

## Your eight steps

|   |                                            |
|---|--------------------------------------------|
| 1 | Choose the depot, stops, and route rules.  |
| 2 | Create distances and route scenarios.      |
| 3 | Build the nearest-neighbour baseline.      |
| 4 | Build the permutation GA.                  |
| 5 | Create the route Product V1 with charts.   |
| 6 | Add traffic, priorities, and time windows. |
| 7 | Compare routes in difficult cases.         |
| 8 | Deploy Product V2 and finish all evidence. |

**Do not jump ahead.** First complete Step 1 below. Do not start the main fuzzy, ANN, or GA code until the team can show the Step 1 evidence.

# Do this first

A beginner team can finish this in three to five working days. Divide the work, but discuss every item together.

## 1. Create these files

- ☐ Create **data/stops.csv**.
- ☐ Create **data/distance\_matrix.csv**.
- ☐ Create **data/README.md**.
- ☐ Create **docs/STEP-1.md**.
- ☐ Create **src/validate\_data.py**.

## 2. Put these columns in the sample data

**Columns:** stop\_id, stop\_name, x\_coordinate, y\_coordinate, priority, earliest\_time, latest\_time

**Starter rules:** Use one depot and 6-10 stops. The distance from a stop to itself must be zero. Explain any one-way distance.

## 3. Write the expected output

visit order, total distance, estimated time, and total penalty or cost

## 4. Write the simple baseline

From the current stop, always visit the nearest stop that has not yet been visited.

## 5. Create exactly 20 sample rows

Begin with only 20 rows. Include normal, boundary, and difficult rows. Your validation program must print the file shape, column names, missing-value count, and invalid values. After this works, you may plan the required 10,000 or more records/scenarios.

## 6. Include these five test cases

- ☐ Ordinary six-stop route
- ☐ Two equally close stops
- ☐ High-priority distant stop
- ☐ Congested connection
- ☐ Impossible or narrow time window

## 7. Make small Git commits

Example commit: **Add campus stops and distance checks**

Every member must make at least one meaningful commit. Do not upload the complete project in one final commit.

## Stop and show this to faculty

|                                                                                           |                                                                                    |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <input type="checkbox"/> Repository works; all members have access.                       | <input type="checkbox"/> STEP-1 explains the problem, inputs, outputs, M1, and M2. |
| <input type="checkbox"/> Field names, units, ranges, and missing-value rules are written. | <input type="checkbox"/> The 20-row sample loads without errors.                   |
| <input type="checkbox"/> One deliberately wrong row is rejected.                          | <input type="checkbox"/> Five named test cases are present.                        |
| <input type="checkbox"/> Baseline rules and a Product V1 screen sketch are included.      | <input type="checkbox"/> Every member can run and explain the validation program.  |

# Smart Irrigation and Crop Water Management System

**In simple words:** Build an app that recommends when and how much to irrigate after looking at soil, weather, rainfall, and crop stage.

## What you must finish

|                                |                                                                                                             |
|--------------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>M1 - Working Product V1</b> | An ANN water predictor or a complete fuzzy irrigation controller, compared with a fixed moisture threshold. |
| <b>M2 - Better Product V2</b>  | Combine ANN prediction with fuzzy decisions, test difficult sensor and weather cases, and deploy the app.   |

## Your eight steps

|   |                                              |
|---|----------------------------------------------|
| 1 | Choose one crop and one growing context.     |
| 2 | Prepare and validate weather and soil data.  |
| 3 | Build the fixed moisture-threshold baseline. |
| 4 | Build the ANN or fuzzy M1 method.            |
| 5 | Create the irrigation Product V1.            |
| 6 | Combine ANN and fuzzy components.            |
| 7 | Compare methods and stress-test sensors.     |
| 8 | Deploy Product V2 and finish all evidence.   |

**Do not jump ahead.** First complete Step 1 below. Do not start the main fuzzy, ANN, or GA code until the team can show the Step 1 evidence.

# Do this first

A beginner team can finish this in three to five working days. Divide the work, but discuss every item together.

## 1. Create these files

- ☐ Create `data/sample_input.csv`.
- ☐ Create `data/README.md`.
- ☐ Create `docs/STEP-1.md`.
- ☐ Create `src/validate_data.py`.

## 2. Put these columns in the sample data

**Columns:** `scenario_id`, `soil_moisture_pct`, `humidity_pct`, `temperature_c`, `rainfall_mm`, `crop_stage`

**Starter rules:** Moisture and humidity 0-100%; rainfall cannot be negative; crop stage initial/development/mid/late.

## 3. Write the expected output

`water_requirement_litre` or `mm`, `irrigation_duration_min`, `decision` off/low/medium/high

## 4. Write the simple baseline

Turn irrigation on when soil moisture is below one fixed threshold. Turn it off after sufficient rain.

## 5. Create exactly 20 sample rows

Begin with only 20 rows. Include normal, boundary, and difficult rows. Your validation program must print the file shape, column names, missing-value count, and invalid values. After this works, you may plan the required 10,000 or more records/scenarios.

## 6. Include these five test cases

- ☐ Dry and hot
- ☐ Adequate moisture
- ☐ Recent heavy rain
- ☐ Dry soil with high humidity
- ☐ Impossible or noisy sensor reading

## 7. Make small Git commits

Example commit: **Define irrigation units and sample scenarios**

Every member must make at least one meaningful commit. Do not upload the complete project in one final commit.

## Stop and show this to faculty

|                                                                                           |                                                                                    |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <input type="checkbox"/> Repository works; all members have access.                       | <input type="checkbox"/> STEP-1 explains the problem, inputs, outputs, M1, and M2. |
| <input type="checkbox"/> Field names, units, ranges, and missing-value rules are written. | <input type="checkbox"/> The 20-row sample loads without errors.                   |
| <input type="checkbox"/> One deliberately wrong row is rejected.                          | <input type="checkbox"/> Five named test cases are present.                        |
| <input type="checkbox"/> Baseline rules and a Product V1 screen sketch are included.      | <input type="checkbox"/> Every member can run and explain the validation program.  |

# Machine Predictive Maintenance and Fault Priority System

**In simple words:** Build a dashboard that reads machine sensor values, estimates fault risk, and tells the maintenance team how urgently the machine needs attention.

## What you must finish

|                                |                                                                                                  |
|--------------------------------|--------------------------------------------------------------------------------------------------|
| <b>M1 - Working Product V1</b> | An ANN fault-risk model or fuzzy priority controller, compared with crisp sensor thresholds.     |
| <b>M2 - Better Product V2</b>  | Combine ANN risk with fuzzy severity, test noisy and extreme readings, and deploy the dashboard. |

## Your eight steps

|   |                                                  |
|---|--------------------------------------------------|
| 1 | Choose the machine observation and fault target. |
| 2 | Prepare and validate cited sensor data.          |
| 3 | Build crisp sensor-threshold rules.              |
| 4 | Build the ANN or fuzzy M1 method.                |
| 5 | Create the maintenance Product V1.               |
| 6 | Combine ANN risk and fuzzy priority.             |
| 7 | Test imbalance, noise, and extreme readings.     |
| 8 | Deploy Product V2 and finish all evidence.       |

**Do not jump ahead.** First complete Step 1 below. Do not start the main fuzzy, ANN, or GA code until the team can show the Step 1 evidence.

# Do this first

A beginner team can finish this in three to five working days. Divide the work, but discuss every item together.

## 1. Create these files

- ☐ Create **data/sample\_input.csv**.
- ☐ Create **data/README.md**.
- ☐ Create **docs/STEP-1.md**.
- ☐ Create **src/validate\_data.py**.

## 2. Put these columns in the sample data

**Columns:** reading\_id, vibration, temperature\_c, rpm, load\_pct, severity, fault\_label

**Starter rules:** Copy vibration and RPM units from one chosen dataset. Do not mix units. Load is 0-100%.

## 3. Write the expected output

fault\_probability, risk\_band (low/medium/high), maintenance\_priority (routine/soon/immediate)

## 4. Write the simple baseline

Use a small set of visible sensor thresholds. Mark temporary thresholds and later justify them with sources or data.

## 5. Create exactly 20 sample rows

Begin with only 20 rows. Include normal, boundary, and difficult rows. Your validation program must print the file shape, column names, missing-value count, and invalid values. After this works, you may plan the required 10,000 or more records/scenarios.

## 6. Include these five test cases

- ☐ Normal operation
- ☐ High vibration only
- ☐ Overheating under high load
- ☐ Several severe signals
- ☐ Missing or noisy sensor value

## 7. Make small Git commits

Example commit: **Document sensor units and fault labels**

Every member must make at least one meaningful commit. Do not upload the complete project in one final commit.

## Stop and show this to faculty

|                                                                                           |                                                                                    |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <input type="checkbox"/> Repository works; all members have access.                       | <input type="checkbox"/> STEP-1 explains the problem, inputs, outputs, M1, and M2. |
| <input type="checkbox"/> Field names, units, ranges, and missing-value rules are written. | <input type="checkbox"/> The 20-row sample loads without errors.                   |
| <input type="checkbox"/> One deliberately wrong row is rejected.                          | <input type="checkbox"/> Five named test cases are present.                        |
| <input type="checkbox"/> Baseline rules and a Product V1 screen sketch are included.      | <input type="checkbox"/> Every member can run and explain the validation program.  |

# Intelligent Loan or Credit Risk Decision Support System

**In simple words:** Build an educational decision-support app that estimates credit risk and explains a recommended action. Use only public, anonymous data.

## What you must finish

|                                |                                                                                           |
|--------------------------------|-------------------------------------------------------------------------------------------|
| <b>M1 - Working Product V1</b> | An ANN risk classifier, compared with a visible score-threshold baseline.                 |
| <b>M2 - Better Product V2</b>  | Use fuzzy logic for borderline cases, check calibration and fairness, and deploy the app. |

## Your eight steps

|   |                                                      |
|---|------------------------------------------------------|
| 1 | Define anonymous inputs, target, and recommendation. |
| 2 | Prepare and validate a public credit dataset.        |
| 3 | Build a transparent score baseline.                  |
| 4 | Build the ANN classifier and show its errors.        |
| 5 | Create the risk Product V1.                          |
| 6 | Add fuzzy handling for borderline cases.             |
| 7 | Compare accuracy, calibration, and fairness.         |
| 8 | Deploy Product V2 and finish all evidence.           |

**Do not jump ahead.** First complete Step 1 below. Do not start the main fuzzy, ANN, or GA code until the team can show the Step 1 evidence.

# Do this first

A beginner team can finish this in three to five working days. Divide the work, but discuss every item together.

## 1. Create these files

- ☐ Create **data/sample\_input.csv**.
- ☐ Create **data/README.md**.
- ☐ Create **docs/STEP-1.md**.
- ☐ Create **src/validate\_data.py**.

## 2. Put these columns in the sample data

**Columns:** record\_id, income, loan\_amount, debt\_to\_income\_pct, repayment\_history, employment\_length, credit\_target

**Starter rules:** State currency and units. Explain every category and missing-value rule. Never collect real financial details from classmates.

## 3. Write the expected output

risk\_band, confidence, recommended\_action (review/approve/reject)

## 4. Write the simple baseline

Give points for a few documented features, add the points, and use visible thresholds for risk bands.

## 5. Create exactly 20 sample rows

Begin with only 20 rows. Include normal, boundary, and difficult rows. Your validation program must print the file shape, column names, missing-value count, and invalid values. After this works, you may plan the required 10,000 or more records/scenarios.

## 6. Include these five test cases

- ☐ Low debt and stable history
- ☐ High debt
- ☐ Borderline score
- ☐ Strong income but poor repayment history
- ☐ Missing or unknown category

## 7. Make small Git commits

Example commit: **Document anonymous credit schema and ethics**

Every member must make at least one meaningful commit. Do not upload the complete project in one final commit.

## Stop and show this to faculty

|                                                                                           |                                                                                    |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <input type="checkbox"/> Repository works; all members have access.                       | <input type="checkbox"/> STEP-1 explains the problem, inputs, outputs, M1, and M2. |
| <input type="checkbox"/> Field names, units, ranges, and missing-value rules are written. | <input type="checkbox"/> The 20-row sample loads without errors.                   |
| <input type="checkbox"/> One deliberately wrong row is rejected.                          | <input type="checkbox"/> Five named test cases are present.                        |
| <input type="checkbox"/> Baseline rules and a Product V1 screen sketch are included.      | <input type="checkbox"/> Every member can run and explain the validation program.  |

# Smart Energy Demand and Appliance Scheduling System

**In simple words:** Build an app that predicts electricity demand and chooses good times to run appliances so that cost and peak load are reduced.

## What you must finish

|                                |                                                                                               |
|--------------------------------|-----------------------------------------------------------------------------------------------|
| <b>M1 - Working Product V1</b> | An ANN forecast or GA scheduler MVP, compared with an original non-optimized schedule.        |
| <b>M2 - Better Product V2</b>  | Combine ANN forecasting with GA scheduling, test tariffs and preferences, and deploy the app. |

## Your eight steps

|   |                                                   |
|---|---------------------------------------------------|
| 1 | Choose one home, lab, or small building.          |
| 2 | Prepare hourly load, tariff, and appliance data.  |
| 3 | Build the original schedule or forecast baseline. |
| 4 | Build the ANN forecast or GA scheduler.           |
| 5 | Create the energy Product V1.                     |
| 6 | Combine ANN forecasting and GA scheduling.        |
| 7 | Compare cost and peaks in difficult cases.        |
| 8 | Deploy Product V2 and finish all evidence.        |

**Do not jump ahead.** First complete Step 1 below. Do not start the main fuzzy, ANN, or GA code until the team can show the Step 1 evidence.

# Do this first

A beginner team can finish this in three to five working days. Divide the work, but discuss every item together.

## 1. Create these files

- ☐ Create **data/hourly\_load.csv**.
- ☐ Create **data/appliances.csv**.
- ☐ Create **data/README.md**.
- ☐ Create **docs/STEP-1.md**.
- ☐ Create **src/validate\_data.py**.

## 2. Put these columns in the sample data

**Columns:** hourly\_load: date, hour, historical\_load\_kw, temperature\_c, tariff\_per\_kwh; appliances: appliance\_id, power\_kw, duration\_slots, earliest\_start, latest\_finish, interruptible, priority

**Starter rules:** Use 24 one-hour slots. Power, energy, tariff, and time-window units must be written clearly.

## 3. Write the expected output

forecast\_load\_kw, appliance start slots, daily\_cost, peak\_load\_kw

## 4. Write the simple baseline

Keep the original user-entered appliance times. If forecasting, use persistence or a simple moving average.

## 5. Create exactly 20 sample rows

Begin with only 20 rows. Include normal, boundary, and difficult rows. Your validation program must print the file shape, column names, missing-value count, and invalid values. After this works, you may plan the required 10,000 or more records/scenarios.

## 6. Include these five test cases

- ☐ Normal tariff
- ☐ Expensive evening peak
- ☐ Narrow appliance time window
- ☐ Two high-power appliances competing
- ☐ Impossible schedule that must be flagged

## 7. Make small Git commits

Example commit: **Add 24-hour energy and appliance fixtures**

Every member must make at least one meaningful commit. Do not upload the complete project in one final commit.

## Stop and show this to faculty

|                                                                                           |                                                                                    |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <input type="checkbox"/> Repository works; all members have access.                       | <input type="checkbox"/> STEP-1 explains the problem, inputs, outputs, M1, and M2. |
| <input type="checkbox"/> Field names, units, ranges, and missing-value rules are written. | <input type="checkbox"/> The 20-row sample loads without errors.                   |
| <input type="checkbox"/> One deliberately wrong row is rejected.                          | <input type="checkbox"/> Five named test cases are present.                        |
| <input type="checkbox"/> Baseline rules and a Product V1 screen sketch are included.      | <input type="checkbox"/> Every member can run and explain the validation program.  |

# Step 1 submission checklist

A team should be able to demonstrate all items below in about three minutes.

- ☐ Repository URL works and all team members are collaborators.
- ☐ Required folders, README.md, and requirements.txt exist.
- ☐ docs/STEP-1.md is complete in simple language.
- ☐ Input/output dictionary includes units and validation rules.
- ☐ data/README.md contains real source URLs and licence information.
- ☐ A valid 20-row sample fixture is present.
- ☐ Five named test cases include a boundary or error case.
- ☐ Baseline pseudocode or rule table is present.
- ☐ Product V1 screen sketch is visible.
- ☐ A script or notebook loads and validates the sample.
- ☐ Every member has made a meaningful commit.
- ☐ Every member can explain the problem, input, output, baseline, M1, and M2.

**Step 1 is not complete** when a team has only copied the problem statement, created slides, downloaded an undocumented dataset, or uploaded all work at once.

## What happens after Step 1 approval?

|   |                                                                                    |
|---|------------------------------------------------------------------------------------|
| 1 | Expand the data plan and create the reproducible 10,000+ record/scenario pipeline. |
| 2 | Implement and measure the simple baseline.                                         |
| 3 | Implement the M1 Soft Computing method.                                            |
| 4 | Join data, baseline, method, output, and visuals into Product V1.                  |
| 5 | Prepare the M1 demo one week before the mid-semester examination.                  |

# Build, test, explain, and show evidence

A strong capstone is not the project with the most complicated words. It is the project that another person can open, run, understand, test, and compare.

## Before every faculty review

- ☐ Pull the latest repository version.
- ☐ Follow your own setup instructions from the beginning.
- ☐ Run one normal case and one error case.
- ☐ Open the latest results and screenshots.
- ☐ Check that every chart has a title, labels, units, and explanation.
- ☐ Ask every member to explain one code part and one result.

## Submission timing

|   |                                                                              |
|---|------------------------------------------------------------------------------|
| 1 | M1 Working Product V1: finish one week before the mid-semester examination.  |
| 2 | M2 Advanced Product V2: finish one week before the end-semester examination. |

## Team notes

---

---

---

---

---

---

---

---

---

---